

AI ASSISTED CODING

LAB ASSIGNMENT - 6

NAME : ROGER A RAJU
BATCH : 13
ROLL NO : 2503A52370

Task - 1 (Classes – Data Validation)

PROMPT:

- Generate a Student class with attributes: name, roll_no, and marks. Add a method is_pass() that returns whether the student has passed (marks $\geq$ 40).

INPUT:

```
class Student:
    """Represents a student with basic attributes and score validation."""

    def __init__(self, name: str, roll_no: int, marks: float):
        if not isinstance(name, str) or not name.strip():
            raise ValueError("Name must be a non-empty string.")

        if not isinstance(roll_no, int) or roll_no <= 0:
            raise ValueError("Roll number must be a positive integer.")

        if not isinstance(marks, (int, float)):
            raise TypeError("Marks must be numeric (int or float).")

        if not (0 <= marks <= 100):
            raise ValueError("Marks must fall between 0 and 100 inclusive.")

        self.name = name.strip()
        self.roll_no = roll_no
        self.marks = float(marks)

    def is_pass(self) -> bool:
        """Determines if student scored the passing threshold (marks >= 40)."""
        return self.marks >= 40

    def __repr__(self) -> str:
        status = "Pass" if self.is_pass() else "Fail"
        return f"Student(name='{self.name}', roll_no={self.roll_no}, marks={self.marks}, status='{status}')"

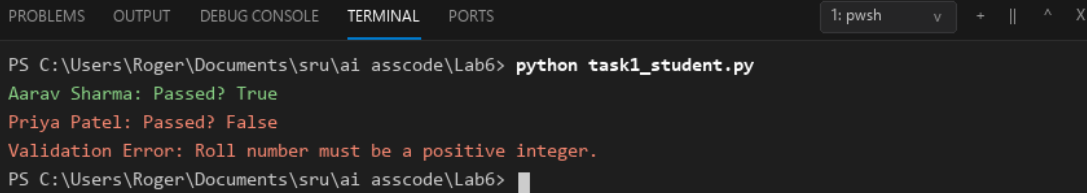
if __name__ == "__main__":
    s1 = Student("Aarav Sharma", 101, 78.5)
    s2 = Student("Priya Patel", 102, 34.0)

    print(f"{s1.name}: Passed? {s1.is_pass()}")
    print(f"{s2.name}: Passed? {s2.is_pass()}")

    try:
        s3 = Student("Rahul", -5, 105)
```

```
except ValueError as e:
    print(f"Validation Error: {e}")
```

OUTPUT:



```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task1_student.py
Aarav Sharma: Passed? True
Priya Patel: Passed? False
Validation Error: Roll number must be a positive integer.
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> █
```

Task - 2 (Loops – Pattern Generation)

PROMPT:

- Generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

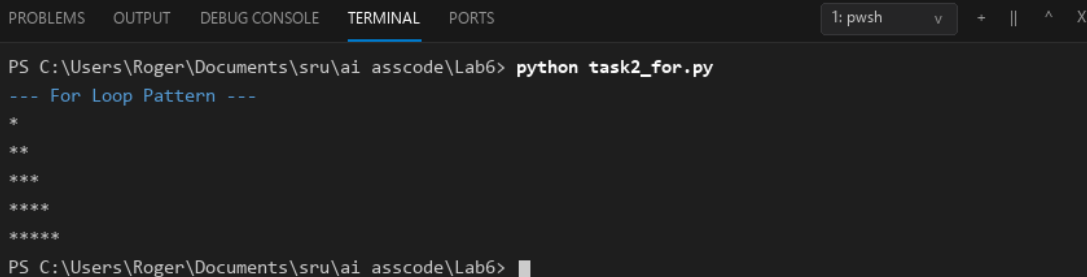
INPUT (by using for loop):

```
def print_triangle_for(n: int) -> None:
    """Prints a right-angled triangle pattern of height n using a for loop."""
    if n <= 0:
        return

    for i in range(1, n + 1):
        print("*" * i)

print("--- For Loop Pattern ---")
print_triangle_for(5)
```

OUTPUT:



```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task2_for.py
--- For Loop Pattern ---
*
**
***
****
*****
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> █
```

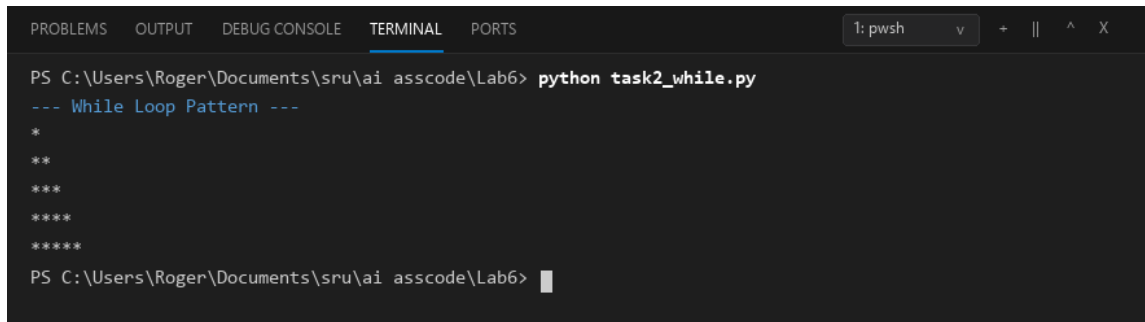
INPUT (by using while loop):

```
def print_triangle_while(n: int) -> None:
    """Prints a right-angled triangle pattern of height n using a while loop."""
    if n <= 0:
        return

    i = 1
    while i <= n:
        print("*" * i)
        i += 1
```

```
print("--- While Loop Pattern ---")
print_triangle_while(5)
```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task2_while.py
--- While Loop Pattern ---
*
**
***
****
*****
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> 
```

Task - 3 (Conditional Statements – Number Analysis)

PROMPT:

- Write a function that checks whether a given number is positive, negative, or zero using if-elif-else. Test the function with multiple inputs.

INPUT:

```
from typing import Union

def analyze_number(num: Union[int, float]) -> str:
    """Classifies a given number as 'Positive', 'Negative', or 'Zero'.

    Args:
        num: A numeric value (int or float).

    Returns:
        A string indicating the sign of the number.
    """
    if not isinstance(num, (int, float)):
        raise TypeError(f"Expected int or float, got {type(num).__name__}")

    if num > 0:
        return "Positive"
    elif num < 0:
        return "Negative"
    else:
        return "Zero"

if __name__ == "__main__":
    test_cases = [15, -42, 0, 3.14159, -0.007, 0.0]

    print(f"{'Input':>10} | {'Classification':<12}")
    print("-" * 28)
    for value in test_cases:
        result = analyze_number(value)
        print(f"{value:>10} | {result:<12}")
```

OUTPUT:

```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task3_number_analysis.py
```

Input	Classification
-------	----------------

15	Positive
-42	Negative
0	Zero
3.14159	Positive
-0.007	Negative
0.0	Zero

```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> █
```

Task - 4 (Nested Conditionals)

PROMT:

- Generate a function `check_discount(age, is_member)` that determines discount eligibility:
- Age $\geq 60 \rightarrow$ Senior discount
- Member $\rightarrow$ Additional discount

Use nested if statements.

INPUT:

```
def check_discount(age: int, is_member: bool) -> str:
    """Determines discount eligibility based on age and membership status

    using nested conditionals.
    """
    if not isinstance(age, int) or age < 0:
        raise ValueError("Age must be a non-negative integer.")

    if not isinstance(is_member, bool):
        raise TypeError("is_member must be a boolean (True/False).")

    if age >= 60:
        if is_member:
            return "Eligible for Senior Discount + Member Bonus (Highest Discount)"
        else:
            return "Eligible for Standard Senior Discount"
    else:
        if is_member:
            return "Eligible for Member Discount"
        else:
            return "No Discount (Standard Rate)"

if __name__ == "__main__":
    test_cases = [
        (65, True),
        (70, False),
        (25, True),
        (30, False),
    ]

    for age, member in test_cases:
        result = check_discount(age, member)
        print(f"Age: {age}, Member: {str(member):<5} -> {result}")
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh v + || ^ X
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task4_nested_conditionals.py
Age: 65, Member: True -> Eligible for Senior Discount + Member Bonus (Highest Discount)
Age: 70, Member: False -> Eligible for Standard Senior Discount
Age: 25, Member: True -> Eligible for Member Discount
Age: 30, Member: False -> No Discount (Standard Rate)
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> █
```

Task - 5 (Class – Mathematical Opera)

PROMPT:

- Create a Circle class with methods to calculate area () and circumference () given the radius.

INPUT:

```
import math
from typing import Union

class Circle:
    """Represents a geometric circle defined by its radius."""

    def __init__(self, radius: Union[int, float]):
        if not isinstance(radius, (int, float)):
            raise TypeError("Radius must be an integer or float.")
        if radius <= 0:
            raise ValueError("Radius must be a positive number greater than zero.")

        self.radius = float(radius)

    def area(self) -> float:
        """Calculates the area: Area = π * r^2."""
        return math.pi * (self.radius ** 2)

    def circumference(self) -> float:
        """Calculates the circumference: C = 2 * π * r."""
        return 2 * math.pi * self.radius

    def __repr__(self) -> str:
        return f"Circle(radius={self.radius})"

if __name__ == "__main__":
    test_radai = [1.0, 5.0, 7.5]

    print(f"{'Radius':>8} | {'Circumference':>15} | {'Area':>15}")
    print("-" * 44)

    for r in test_radai:
        c = Circle(r)
        print(f"{c.radius:8.2f} | {c.circumference():15.4f} | {c.area():15.4f}")
```

OUTPUT:

```
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task5_circle.py
Radius | Circumference | Area
-----|-----|-----
1.00 | 6.2832 | 3.1416
5.00 | 31.4159 | 78.5398
7.50 | 47.1239 | 176.7146
PS C:\Users\Roger\Documents\sru\ai asscode\Lab6>
```